



Supabase Database Schema Analysis

Plataforma de Gestion de Canchas Deportivas

Av. Bolivar C1, San Sebastian, Cusco

Documento:	Analisis de Tablas para Supabase
Version:	1.0
Base de datos:	Supabase (PostgreSQL)
Total de tablas:	7 tablas principales + 3 vistas + 2 funciones
Fecha:	Mayo 2026

1. Resumen Ejecutivo

Este documento presenta el analisis completo de la estructura de base de datos necesaria para migrar el sistema CREARD desde SQLite (Prisma) a Supabase (PostgreSQL). La plataforma CREARD es un sistema de gestion de canchas deportivas ubicado en Av. Bolivar C1, San Sebastian, Cusco, que administra 6 canchas (4 de futbol 5 y 2 de voley) con un flujo completo de reservas, pagos y control financiero.

La migracion a Supabase ofrece ventajas significativas respecto a SQLite: autenticacion integrada con JWT y Row Level Security (RLS), base de datos PostgreSQL en la nube con alta disponibilidad, API REST automatica (PostgREST), subscriptions en tiempo real, almacenamiento de archivos (Storage), y funciones edge para logica del lado del servidor. Ademias, Supabase utiliza tipos de datos nativos de PostgreSQL como ENUM, JSONB, UUID, TIMESTAMPTZ y NUMERIC, que proporcionan mayor integridad, rendimiento y seguridad de los datos en comparacion con las soluciones basadas en SQLite.

El esquema propuesto incluye 7 tablas principales (branches, courts, users, bookings, payments, expenses, reviews), 10 tipos ENUM para garantizar la integridad referencial, 3 vistas de analisis financiero, 2 funciones utilitarias para disponibilidad y reportes, triggers automaticos para timestamps, y politicas RLS para seguridad granular por rol (admin/user). Se incluyen ademias constraints de validacion de datos, indices optimizados para las consultas mas frecuentes, y datos seed iniciales para las 6 canchas y la sucursal principal.

Componente	Cantidad	Descripcion
Tablas	7	branches, courts, users, bookings, payments, expenses, reviews
ENUMs	10	user_role, booking_status, slot_status, payment_method, payment_status, payment_type, expense_category, court_sport
Vistas	3	v_daily_summary, v_court_revenue_ranking, v_today_status
Funciones	2	get_court_availability, get_financial_summary
Triggers	6	Auto-update de updated_at en todas las tablas principales
Políticas RLS	15+	Seguridad granular por rol (admin/user) y por operacion (SELECT/INSERT/UPDATE/DELETE)
Indices	18+	Indices optimizados para busquedas por fecha, estado, usuario, cancha

Tabla 1. Resumen general del esquema de base de datos

2. Mejoras Respecto a SQLite (Prisma)

La migracion de SQLite a PostgreSQL mediante Supabase permite aprovechar las capacidades avanzadas de un motor de base de datos relacional de nivel empresarial. A continuacion se detallan las

principales mejoras implementadas en el esquema para Supabase, cada una de las cuales aporta beneficios concretos en terminos de integridad, rendimiento y seguridad de la aplicacion CREARD.

2.1 Tipos de Datos Nativos de PostgreSQL

SQLite utiliza tipado dinamico y no soporta tipos nativos como ENUM, JSONB, UUID o TIMESTAMPTZ. En el esquema original de Prisma, los campos de estado (status, role, method, etc.) se almacenaban como cadenas de texto (String), lo que permite cualquier valor arbitrario y carece de validacion a nivel de base de datos. En Supabase, se definen 10 tipos ENUM que restringen los valores permitidos directamente en PostgreSQL, garantizando que no se inserten valores invalidos como "status=xyz" o "role=superadmin". Los campos JSON (images, amenities) se migran de String a JSONB, permitiendo consultas y indexacion sobre contenido JSON de manera nativa, como buscar canchas que tengan "Wi-Fi" en sus amenidades o filtrar por URLs de imagenes especificas.

Campo	SQLite (Prisma)	Supabase (PostgreSQL)	Ventaja
ID primaria	String (cuid)	UUID (gen_random_uuid)	Estandar universal, mejor performance en indices
Estado reserva	String	booking_status (ENUM)	Validacion nativa, sin valores invalidos
Imagenes	String (JSON manual)	JSONB	Consultas e indices sobre contenido JSON
Amenidades	String (JSON manual)	JSONB	Filtros y busquedas nativas en JSON
Fechas	DateTime o String	TIMESTAMPTZ	Zona horaria automatica, mejor precision
Precio	Float	NUMERIC(10,2)	Precision decimal exacta (sin errores de punto flotante)
Horas	String ("18:00")	TIME	Operaciones aritmeticas y comparaciones nativas

Tabla 2. Comparacion de tipos de datos entre SQLite y Supabase

2.2 Row Level Security (RLS)

SQLite no tiene soporte nativo para politicas de seguridad a nivel de fila. En el esquema Prisma original, toda la logica de autorizacion se realizaba en el backend de Next.js, lo que implica que cada endpoint API debia verificar manualmente los permisos del usuario. Con Supabase RLS, las politicas de acceso se definen directamente en la base de datos y se ejecutan de forma automatica en cada consulta, independientemente de como se acceda a los datos (API REST, SDK, o directamente desde el frontend con el cliente Supabase). Esto elimina la posibilidad de que un usuario acceda a datos de otro usuario mediante una peticion maliciosa, ya que PostgreSQL rechaza la consulta antes de devolver cualquier resultado.

El esquema define 15+ politicas RLS que cubren los siguientes casos: los clientes pueden leer y modificar unicamente sus propias reservas, pagos y perfil; los administradores tienen acceso completo a todas las tablas para gestionar reservas, gastos, usuarios y pagos; las canchas y sucursales son de lectura publica para usuarios anonimos y autenticados; las reseñas son visibles para todos pero solo pueden ser creadas por usuarios autenticados para su propia experiencia; y los gastos son accesibles unicamente para administradores, protegiendo la informacion financiera sensible del negocio.

2.3 Constraints y Validacion de Datos

PostgreSQL permite definir restricciones CHECK a nivel de tabla que validan los datos antes de insertarlos o actualizarlos, algo que SQLite soporta de forma limitada. En el esquema de Supabase se implementan las siguientes restricciones: el precio total de una reserva no puede ser negativo (chk_total_price); el monto del adelanto no puede superar el precio total (chk_advance); la suma del adelanto mas el saldo restante debe ser exactamente igual al precio total (chk_amounts); la hora de fin de una reserva debe ser posterior a la hora de inicio (chk_times); los montos de pago y gastos deben ser valores positivos (chk_payment_amount, chk_expense_amount); y las calificaciones de reseñas deben estar entre 1 y 5 estrellas (CHECK en la columna rating). Estas restricciones garantizan la integridad de los datos incluso si la aplicacion frontend tiene errores de validacion.

3. Detalle de Tablas

3.1 branches (Sucursales)

La tabla branches almacena las sucursales o locales de CREARD. Aunque actualmente la plataforma opera desde una sola ubicacion en Av. Bolivar C1, San Sebastian, Cusco, incluir esta tabla desde el inicio permite escalar el negocio a multiples sedes en el futuro sin requerir cambios estructurales en la base de datos. Cada cancha pertenece a una sucursal mediante la relacion FOREIGN KEY branch_id, lo que facilita la gestion independiente de horarios, precios y disponibilidad por ubicacion. Los campos incluyen nombre, direccion, ciudad, telefono, email y un flag is_active para desactivar sucursales temporalmente sin eliminar sus datos historicos.

Columna	Tipo	Restriccion	Descripcion
id	UUID	PK	Identificador unico
name	TEXT	NOT NULL	Nombre de la sucursal
address	TEXT	NOT NULL	Direccion fisica
city	TEXT	DEFAULT 'Cusco'	Ciudad de ubicacion
phone	TEXT	NULL	Telefono de contacto
email	TEXT	NULL	Email de contacto
is_active	BOOLEAN	DEFAULT true	Sucursal operativa

created_at	TIMESTAMPTZ	NOT NULL	Fecha de creacion
updated_at	TIMESTAMPTZ	NOT NULL	Fecha de actualizacion (trigger)

Tabla 3. Estructura de la tabla branches

3.2 courts (Canchas)

La tabla courts es el nucleo del sistema, almacenando las 6 canchas deportivas: 4 de futbol 5 (Cancha de Futbol 1, 2, 3 y 4) y 2 de voley (Cancha de Voley A y B). Cada cancha tiene un tipo de deporte definido por el ENUM court_sport, un precio por hora en soles (NUMERIC(10,2) para precision decimal exacta), y campos JSONB para imagenes y amenidades que permiten almacenar arrays de URLs y listas de servicios con la posibilidad de consultarlos directamente desde PostgreSQL. El campo is_active permite desactivar temporalmente una cancha (por mantenimiento, por ejemplo) sin eliminar sus reservas historicas. Los precios varian entre S/30 y S/55 por hora segun las caracteristicas de cada cancha, siendo las canchas techadas (Futbol 3) y las de futbol premium (Futbol 2, Futbol 4) las mas costosas.

Columna	Tipo	Restriccion	Descripcion
id	UUID	PK	Identificador unico
name	TEXT	NOT NULL	Nombre de la cancha
sport	court_sport	NOT NULL	ENUM: futbol / voley
description	TEXT	NULL	Descripcion detallada
branch_id	UUID	FK branches	Sucursal a la que pertenece
images	JSONB	DEFAULT '[]'	Array de URLs de imagenes
price_per_hour	NUMERIC(10,2)	NOT NULL	Precio por hora en soles
is_active	BOOLEAN	DEFAULT true	Cancha disponible
amenities	JSONB	DEFAULT '[]'	Array de amenidades/servicios

Tabla 4. Estructura de la tabla courts

3.3 users (Usuarios)

La tabla users almacena los dos roles del sistema: administradores (admin) y clientes (user). Se definen mediante el ENUM user_role que solo permite estos dos valores, previniendo la creacion de roles no autorizados. El campo password almacena el hash bcrypt de la contrasena, nunca la contrasena en texto plano. Se recomienda utilizar Supabase Auth para la autentificacion real, manteniendo esta tabla como perfil extendido vinculado al ID de Supabase Auth mediante auth.uid(). El email tiene la restriccion UNIQUE para evitar duplicados y se indexa para acelerar las busquedas durante el inicio de sesion. Los campos is_active y updated_at permiten desactivar usuarios y rastrear la ultima modificacion de cada perfil.

Columna	Tipo	Restriccion	Descripcion
id	UUID	PK	Identificador unico (auth.uid())
name	TEXT	NOT NULL	Nombre completo
email	TEXT	UNIQUE, NOT NULL	Correo electronico
phone	TEXT	NULL	Numero de telefono
password	TEXT	NOT NULL	Hash bcrypt de la contraseña
role	user_role	DEFAULT 'user'	ENUM: user / admin
is_active	BOOLEAN	DEFAULT true	Usuario activo

Tabla 5. Estructura de la tabla users

3.4 bookings (Reservas)

La tabla bookings es la mas compleja del esquema, gestionando el ciclo completo de vida de una reserva con dos campos de estado independientes: status (booking_status) visible para el cliente y slot_status (slot_status) visible para el administrador. El flujo de una reserva sigue el siguiente ciclo: el cliente crea una reserva en estado pending con slot_status available; el admin confirma la reserva cambiando status a confirmed y slot_status a reserved; el cliente realiza el adelanto del 50% cambiando status a partially_paid; al completar el pago restante el status cambia a fully_paid; durante el partido el slot_status cambia a in_play; al finalizar pasa a finished y el booking_status a completed. Si el cliente no paga a tiempo, la reserva expira (expired). Si no se presenta, se marca como no_show. El admin puede cancelar en cualquier momento o bloquear slots para mantenimiento.

Los constraints de validacion garantizan que: el precio total no sea negativo, el adelanto no supere el total, la suma de adelanto y restante iguale el total, y la hora de fin sea posterior a la hora de inicio. Se definen indices compuestos para optimizar las busquedas mas frecuentes, como buscar reservas por cancha y fecha, y un indice UNIQUE para prevenir reservas duplicadas en el mismo slot horario de la misma cancha.

Columna	Tipo	Restriccion	Descripcion
id	UUID	PK	Identificador unico
court_id	UUID	FK courts	Cancha reservada
user_id	UUID	FK users	Cliente que reserva
date	DATE	NOT NULL	Fecha de la reserva (YYYY-MM-DD)
start_time	TIME	NOT NULL	Hora de inicio (HH:MM)
end_time	TIME	NOT NULL	Hora de fin (HH:MM)
total_price	NUMERIC(10,2)	NOT NULL, CHECK>=0	Precio total en soles
advance_amount	NUMERIC(10,2)	DEFAULT 0	Monto del adelanto (50%)

remaining_amount	NUMERIC(10,2)	DEFAULT 0	Saldo pendiente (50%)
status	booking_status	DEFAULT 'pending'	Estado visible para el cliente
slot_status	slot_status	DEFAULT 'available'	Estado del slot para el admin
payment_method	payment_method	NULL	Metodo de pago (ENUM)
notes	TEXT	NULL	Notas adicionales

Tabla 6. Estructura de la tabla bookings

3.5 payments (Pagos)

La tabla payments registra cada transaccion financiera individual, vinculada a una reserva y a un usuario. Cada pago tiene un tipo (advance para el adelanto del 50%, remaining para el saldo restante, o full para pagos completos del 100%), un metodo de pago (Yape, Plin, Culqi, tarjeta, efectivo o transferencia definidos por el ENUM payment_method), y un estado (pending, completed, failed o refunded). El campo external_ref almacena la referencia del medio de pago externo, como los codigos YAP-001, PLN-001 o CUL-001 que permiten conciliar los pagos con las plataformas externas. Los indices sobre booking_id, user_id, status, method y created_at permiten generar reportes financieros eficientes agrupados por cualquier combinacion de criterios.

Columna	Tipo	Restriccion	Descripcion
id	UUID	PK	Identificador unico
booking_id	UUID	FK bookings	Reserva asociada
user_id	UUID	FK users	Usuario que paga
amount	NUMERIC(10,2)	NOT NULL, CHECK>0	Monto del pago
type	payment_type	NOT NULL	ENUM: advance / remaining / full
method	payment_method	NOT NULL	ENUM: yape / plin / culqi / card / cash / transfer
status	payment_status	DEFAULT 'pending'	ENUM: pending / completed / failed / refunded
external_ref	TEXT	NULL	Referencia externa (YAP-001, etc.)

Tabla 7. Estructura de la tabla payments

3.6 expenses (Gastos)

La tabla expenses registra los gastos operativos del negocio, categorizados mediante el ENUM expense_category que incluye: mantenimiento (reparacion de cesp ed, iluminacion, instalaciones), servicios (energia electrica, agua, internet), personal (sueldos de limpieza y mantenimiento), alquiler (canon del terreno), y otros (materiales de limpieza, insumos varios). Cada gasto tiene un monto positivo validado por la constraint chk_expense_amount, una fecha para agrupacion temporal en

reportes, y campos de descripción y notas para detalle adicional. Los índices sobre `category` y `date` permiten generar reportes de gastos por periodo y por categoría de forma eficiente.

Columna	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
description	TEXT	NOT NULL	Descripción del gasto
amount	NUMERIC(10,2)	NOT NULL, CHECK>0	Monto del gasto en soles
category	expense_category	NOT NULL	ENUM: mantenimiento / servicios / personal / alquiler / otros
date	DATE	NOT NULL	Fecha del gasto
notes	TEXT	NULL	Notas adicionales

Tabla 8. Estructura de la tabla expenses

3.7 reviews (Resenas)

La tabla reviews almacena las calificaciones y comentarios de los clientes sobre las canchas. Cada resena tiene una calificación de 1 a 5 estrellas (validada por la constraint CHECK en la columna rating) y un comentario opcional. La constraint UNIQUE(user_id, court_id) garantiza que un usuario solo pueda dejar una resena por cancha, evitando duplicados y manipulación de calificaciones. Las resenas son visibles para todos los usuarios (anonimos y autenticados) pero solo los usuarios autenticados pueden crear resenas, y unicamente para canchas donde hayan tenido al menos una reserva completada. Los índices sobre `court_id` permiten calcular el promedio de calificaciones por cancha de forma eficiente.

Columna	Tipo	Restricción	Descripción
id	UUID	PK	Identificador único
court_id	UUID	FK courts	Cancha reseada
user_id	UUID	FK users	Usuario que resena
rating	SMALLINT	CHECK 1-5	Calificación (1 a 5 estrellas)
comment	TEXT	NULL	Comentario del usuario
created_at	TIMESTAMPZ	NOT NULL	Fecha de la resena

Tabla 9. Estructura de la tabla reviews

4. Tipos ENUM Definidos

Los tipos ENUM de PostgreSQL garantizan la integridad referencial a nivel de base de datos, impidiendo la inserción de valores no válidos en columnas de estado, rol, método de pago y demás campos categoricos. A diferencia de SQLite donde estos campos son simples cadenas de texto,

PostgreSQL valida cada valor contra los permitidos antes de aceptar la insercion o actualizacion, lo que elimina errores de validacion en la aplicacion y proporciona documentacion implicita del dominio de cada campo.

ENUM	Valores Permitidos	Usado en
user_role	user, admin	users.role
booking_status	pending, confirmed, partially_paid, fully_paid, completed, cancelled, no_show, expired	bookings.status
slot_status	available, reserved, in_play, finished, blocked, expired	bookings.slot_status
payment_method	yape, plin, culqi, card, cash, transfer	bookings.payment_method, payments.method
payment_status	pending, completed, failed, refunded	payments.status
payment_type	advance, remaining, full	payments.type
expense_category	mantenimiento, servicios, personal, alquiler, otros	expenses.category
court_sport	futbol, voley	courts.sport

Tabla 10. Tipos ENUM y sus valores permitidos

5. Vistas y Funciones Utilitarias

5.1 Vistas de Analisis

Se incluyen 3 vistas predefinidas que simplifican las consultas mas comunes del modulo financiero y administrativo del sistema CREARD. Estas vistas encapsulan la logica de joins y agregaciones, permitiendo obtener informacion consolidada con una simple consulta SELECT sin necesidad de escribir queries complejas en el frontend o backend.

La vista `v_daily_summary` proporciona un resumen financiero diario que incluye: total de reservas, reservas completadas, ingresos totales (suma de pagos completados), gastos totales y balance neto (ingresos menos gastos). Es ideal para el dashboard principal del administrador y para generar reportes semanales o mensuales agrupando por fecha.

La vista `v_court_revenue_ranking` muestra un ranking de canchas ordenadas por ingresos totales, incluyendo el total de reservas, el promedio de calificacion y la cantidad de reseñas. Permite identificar rapidamente cuales canchas son las mas rentables y mejor valoradas por los clientes, facilitando decisiones sobre inversiones en mantenimiento o promociones.

La vista `v_today_status` muestra el estado actual de todas las canchas para el dia de hoy, incluyendo las reservas activas con datos del cliente, horarios, precios, montos pagados y metodo de pago. Es la vista principal que utiliza el panel de control del administrador para visualizar la operacion del dia y

gestionar los slots en tiempo real.

5.2 Funciones Utilitarias

Se incluyen 2 funciones PostgreSQL marcadas como SECURITY DEFINER (se ejecutan con los privilegios del creador, no del invocador) para operaciones críticas que requieren acceso a datos que el usuario no podría consultar directamente con RLS activo.

La funcion `get_court_availability(p_court_id UUID, p_date DATE)` genera automaticamente los slots horarios de una hora desde las 08:00 hasta las 22:00 para una cancha y fecha especificas, realizando un LEFT JOIN con las reservas existentes para indicar cuales slots estan disponibles y cuales estan ocupados, incluyendo el nombre del cliente y el estado del slot. Esto simplifica enormemente la logica del frontend para mostrar la grilla de disponibilidad.

La funcion `get_financial_summary(p_start_date DATE, p_end_date DATE)` calcula el resumen financiero en un rango de fechas: ingresos totales, gastos totales, balance neto, ingresos pendientes, total de reservas y reservas completadas. Es la base del modulo de reportes financieros que permite al administrador analizar la rentabilidad del negocio por periodo.

6. Diagrama de Relaciones (ER)

El siguiente diagrama muestra las relaciones entre las 7 tablas principales del esquema. Las flechas indican las foreign keys con sus acciones ON DELETE CASCADE, lo que significa que al eliminar un registro padre se eliminan automaticamente todos sus registros hijos asociados, garantizando que no queden datos huérfanos en la base de datos.

Tabla Origen	Relacion	Tabla Destino	Cardinalidad
branches	1 : N	courts	Una sucursal tiene muchas canchas
courts	1 : N	bookings	Una cancha tiene muchas reservas
courts	1 : N	reviews	Una cancha tiene muchas resenas
users	1 : N	bookings	Un usuario tiene muchas reservas
users	1 : N	payments	Un usuario tiene muchos pagos
users	1 : N	reviews	Un usuario tiene muchas resenas
bookings	1 : N	payments	Una reserva tiene muchos pagos

Tabla 11. Relaciones entre tablas (Diagrama ER)

7. Recomendaciones de Implementacion

7.1 Autenticacion con Supabase Auth

Se recomienda utilizar Supabase Auth como sistema de autenticacion principal en lugar de gestionar las contraseñas manualmente. Supabase Auth proporciona registro, login, recuperacion de contraseña, verificacion de email y autenticacion con proveedores sociales (Google, Facebook, etc.) de forma integrada. El ID del usuario autenticado (`auth.uid()`) se vincula a la tabla `users` como clave primaria, permitiendo que las políticas RLS funcionen de forma nativa. La columna `password` puede eliminarse de la tabla `users` si se utiliza Supabase Auth exclusivamente, o mantenerse como respaldo si se requiere un sistema de autenticacion paralelo.

7.2 Almacenamiento de Imagenes con Supabase Storage

Las imagenes de las canchas pueden almacenarse directamente en Supabase Storage en lugar de referenciar URLs externas. Supabase Storage proporciona un bucket de almacenamiento con control de acceso, transformacion de imagenes on-the-fly (resize, crop, formato) y CDN integrado para entrega rapida. Se recomienda crear un bucket "court-images" con acceso publico de lectura y restringir la escritura a usuarios autenticados con rol admin.

7.3 Realtime Subscriptions

Supabase Realtime permite recibir actualizaciones en tiempo real cuando se modifican las tablas de `bookings` o `slot_status`. Esto es especialmente util para el panel del administrador, donde multiples operadores pueden estar gestionando reservas simultaneamente y necesitan ver los cambios de otros en tiempo real sin refrescar la pagina. Tambien se puede utilizar para notificar a los clientes cuando su reserva cambia de estado (confirmada, pagada, etc.).

7.4 Migracion del Schema

El archivo SQL adjunto (`creard_supabase_schema.sql`) contiene todo el esquema listo para ejecutar en el SQL Editor de Supabase. Los pasos para la migracion son: crear un nuevo proyecto en supabase.com, copiar el contenido del archivo SQL y ejecutarlo en el SQL Editor, verificar que todas las tablas, enums, triggers, vistas y funciones se hayan creado correctamente, ejecutar las sentencias `INSERT` de datos seed para las 6 canchas y la sucursal principal, y finalmente configurar los buckets de Storage para las imagenes si se desea utilizar el almacenamiento nativo de Supabase.

7.5 Adaptacion del Frontend

Para adaptar el frontend Next.js existente a Supabase, se recomienda instalar el SDK de Supabase (`@supabase/supabase-js`), crear un cliente Supabase con las credenciales del proyecto, reemplazar las llamadas a la API Prisma por consultas directa al cliente Supabase o mantener los endpoints API de Next.js como capa intermedia, actualizar el store de Zustand para utilizar el auth state de Supabase, y modificar los componentes `AdminDashboard`, `BookingView` y demas para utilizar los nuevos tipos `ENUM` de PostgreSQL. La transicion puede ser gradual, manteniendo la API de Next.js como

intermediaria y reemplazando Prisma por el cliente Supabase en el backend.